

Relatório de Vulnerabilidades e Mitigações de Segurança

Sweet Spot - Treinamento Solo de Precisão no Tênis

Redes de Computadores e Cibersegurança - 6º semestre

Este relatório foi elaborado com base no documento oficial do Projeto Interdisciplinar e no resumo técnico apresentado. Os achados classificados como potenciais são uma análise de ameaça/arquitetura. A seção de evidências apresenta controles propostos de testes a serem efetivamente realizados.

1. Objetivo e escopo

- Avaliar os principais riscos de segurança da solução Sweet Spot considerando aplicativo, API/backend, banco de dados, processamento por câmeras, Arduino/IoT, comunicação e dados de sessões de treino.
- O documento atende ao requisito da disciplina de Redes de Computadores e Cibersegurança de entregar relatório de vulnerabilidades do sistema e do código, com mitigação aplicada e evidências de testes.
- O projeto acadêmico exige autenticação, autorização por perfil, proteção de credenciais, validação de entradas, proteção contra riscos do OWASP Top 10, comunicação cifrada, menor privilégio, identificação de dispositivos, proteção de chaves/tokens, bloqueio de comandos indevidos, logs, segmentação quando aplicável e parada segura.

2. Contexto técnico da Sweet Spot

- A solução realiza treinamento solo de precisão no tênis. No protótipo atual, bolinhas de isopor representam as bolas que seriam lançadas por uma máquina. Duas webcams apoiam a detecção da área de treino, alvos e movimento das bolinhas; Arduino e LEDs compõem a parte física e de feedback.
- O fluxo lógico considerado neste relatório é: câmera/sensor → processamento local → evento de treino → lógica de pontuação → feedback por LEDs → registro da sessão → API/backend → persistência e consulta.
- O documento oficial também prevê, em sua arquitetura-alvo, aplicativo mobile, site/dashboard, API, dispositivo IoT/simulador e integração com uma plataforma mãe. A comunicação deve ser documentada e segura.

3. Ativos que precisam ser protegidos

- Identidade e credenciais de jogadores, treinadores/autores e administradores.
- Histórico de sessões, pontuação, precisão, tempo de reação e telemetria.
- Definições de treinos, regras de pontuação, versões e parâmetros.
- Tokens/chaves usados pela API e pelo dispositivo IoT.
- Integridade dos comandos enviados ao dispositivo e dos eventos produzidos pelos sensores/câmeras.
- Disponibilidade do processamento de câmera, API e sessão de treino.
- Dados de menores, quando houver esse público, que recebem proteção reforçada conforme o escopo do projeto.

4. Modelo de ameaça

- Adversário de rede local: tenta acessar a API, capturar credenciais/tokens ou enviar comandos falsos.

- Usuário autenticado com privilégios insuficientes: tenta consultar ou alterar sessões, treinos ou funções administrativas de outro usuário.
- Cliente malicioso: envia JSON, IDs, métricas ou parâmetros fora do formato esperado para explorar validações ausentes.
- Atacante com acesso físico ao protótipo: tenta manipular USB/serial, câmera, Arduino, fonte ou conexões.
- Entrada multimídia não confiável: frames de câmera ou arquivos podem causar consumo excessivo de CPU/memória se forem processados sem limites.
- Dependência comprometida ou desatualizada: biblioteca de visão computacional, API ou firmware pode introduzir vulnerabilidade.

5. Matriz de vulnerabilidades e riscos

ID	Vulnerabilidade / risco	Impacto	Prob.	Prioridade	Mitigação
V01	API sem autenticação forte	Acesso indevido a sessões/treinos	Média	Alta	Bearer/token seguro + TLS + expiração/rotação
V02	Falha de autorização por objeto (IDOR/BOLA)	Acesso a dados de outro jogador	Média	Alta	Checagem server-side de ownership/papel
V03	Mass assignment / campos privilegiados	Alteração de role, score ou publicação	Média	Alta	Allowlist de propriedades + DTOs
V04	Injeção por entrada não validada	Manipulação de consultas/comandos	Média	Alta	Validação + queries parametrizadas + whitelist
V05	Abuso de recursos da API	DoS / indisponibilidade	Média	Média	Payload limit + rate limit + paginação
V06	Comandos IoT não autenticados	Alteração indevida do protótipo	Média	Crítica	Identidade do dispositivo + whitelist + estado seguro
V07	Segredos hardcoded	Comprometimento de API/IoT	Média	Alta	Variáveis de ambiente/secret manager
V08	Processamento excessivo de câmera	Travamento/DoS local	Média	Média	Limite FPS/resolução + timeouts
V09	Logs com dados sensíveis	Vazamento de credenciais/PII	Baixa	Média	Redação de segredos + minimização
V10	Dependências vulneráveis	Execução/comprometimento indireto	Média	Alta	SCA + atualização e inventário
V11	Má configuração de produção	Exposição de debug/endpoints	Média	Alta	debug=False + hardening + inventário
V12	Falha de parada segura	Risco físico e operacional	Baixa	Crítica	Estado FAIL-SAFE + bloqueio até reset explícito

6. Mitigações de segurança aplicadas/propostas

- Autenticação: utilizar tokens ou sessão segura, nunca credenciais hardcoded no código-fonte. Segredos devem vir de variáveis de ambiente ou gerenciador de segredos.
- Autorização: separar explicitamente jogador, treinador/autor e administrador. A autorização deve ser verificada no servidor em cada operação sensível, não apenas no frontend.

- Validação de entrada: aceitar somente campos esperados, limitar tamanho de strings e valores numéricos, rejeitar tipos inesperados e não executar conteúdo recebido como comando.
- API: usar HTTPS/TLS em ambiente de rede, versionar endpoints, limitar taxa de requisições e registrar falhas de autenticação e ações administrativas.
- IoT: atribuir identidade ao dispositivo, usar token/chave protegido, aceitar apenas comandos de uma lista permitida e colocar o dispositivo em estado seguro quando a comunicação ou sensor estiver inconsistente.
- Câmeras: limitar resolução/FPS processados, validar índice de câmera e tratar falhas sem derrubar o restante do sistema.
- Banco: usar consultas parametrizadas/ORM, conta de banco com menor privilégio e separação entre leitura e escrita quando viável.
- Logs: registrar autenticação, publicação, comandos, falhas e eventos críticos sem registrar senhas, tokens ou imagens desnecessárias.
- Disponibilidade: aplicar limites de tamanho, rate limiting, timeouts, filas/buffer quando necessário e degradação segura.
- Parada segura: comandos de emergência e falhas críticas devem interromper a sessão e bloquear novos comandos até reinicialização explícita.

7. Evidências de segurança

- Foi criado um conjunto de código de referência com controles de autenticação, autorização por papel, validação de JSON, limite de corpo da requisição, rate limiting simples e tratamento de erros.
- Também foi criado um wrapper para entrada de webcam com validação do índice, limite defensivo de resolução e controle de FPS.
- Foram preparados testes automatizados para demonstrar que endpoints sem autenticação são rejeitados e que funções administrativas exigem papel adequado.
- Importante: esses testes são evidências dos controles de referência produzidos para a entrega. Eles não comprovam que o código real da equipe esteja seguro sem que o código-fonte real seja submetido à mesma bateria.

8. Plano de testes de segurança

- Teste 01 - autenticação: chamar endpoint protegido sem Authorization e confirmar HTTP 401.
- Teste 02 - autorização: usar credencial de jogador para endpoint administrativo e confirmar HTTP 403.
- Teste 03 - validação: enviar campos desconhecidos, tipos incorretos, números negativos e strings excessivamente longas; a API deve rejeitar a entrada.
- Teste 04 - volume: enviar requisições repetidas e confirmar aplicação do limite de taxa.
- Teste 05 - comandos IoT: tentar enviar comando fora da whitelist; o dispositivo deve rejeitar sem alterar o estado.
- Teste 06 - falha de câmera: desconectar uma webcam durante a execução; o sistema deve registrar a falha e evitar travamento inseguro.
- Teste 07 - logs: verificar que senhas, tokens e conteúdo sensível não aparecem em logs.
- Teste 08 - dependências: executar scanner de dependências e corrigir vulnerabilidades conhecidas antes da entrega.
- Teste 09 - API: usar OWASP ZAP ou ferramenta equivalente em ambiente local/de testes para identificar falhas de configuração, autenticação e validação.
- Teste 10 - parada segura: interromper a comunicação/sensor durante uma sessão e verificar que o sistema entra em estado seguro.

9. Mapeamento para OWASP

- A01 - Broken Access Control: mitigado por autorização por papel e checagem de acesso no backend.

- A02 - Cryptographic Failures: mitigado por TLS em trânsito e não armazenamento de credenciais em texto puro.
- A03 - Injection: mitigado por validação de entrada e consultas parametrizadas.
- A04 - Insecure Design: mitigado pela definição de estados seguros, whitelist de comandos e parada segura.
- A05 - Security Misconfiguration: mitigado desativando debug em produção, configurando limites e separando segredos.
- A06 - Vulnerable and Outdated Components: mitigado por inventário e scanner periódico de dependências.
- A07 - Identification and Authentication Failures: mitigado por autenticação centralizada, tokens protegidos e expiração/rotação quando aplicável.
- A08 - Software and Data Integrity Failures: mitigado por versionamento, revisão de código e validação de dados recebidos.
- A09 - Security Logging and Monitoring Failures: mitigado por logs estruturados e auditoria de ações críticas.
- A10 - SSRF: relevante caso o backend busque URLs fornecidas pelo usuário; nesse cenário, usar allowlist e bloquear destinos internos. Se a API não fizer requisições remotas controladas pelo usuário, marcar como não aplicável.

10. Riscos específicos de API

- API1 Broken Object Level Authorization: endpoints que recebem session_id, training_id ou user_id precisam verificar se o objeto pertence ao usuário ou se o papel permite acesso.
- API2 Broken Authentication: evitar tokens previsíveis, credenciais padrão e armazenamento inseguro.
- API3 Broken Object Property Level Authorization: impedir que o cliente altere campos privilegiados como role, owner_id, published ou score oficial.
- API4 Unrestricted Resource Consumption: aplicar limites de payload, paginação, rate limiting e limites de processamento.
- API5 Broken Function Level Authorization: funções de publicação, gestão administrativa e dispositivos devem exigir papel adequado.
- API8 Security Misconfiguration: remover debug, endpoints de teste e configurações padrão antes do deploy.
- API9 Improper Inventory Management: manter documentação OpenAPI e versionamento, incluindo endpoints ativos e descontinuados.
- API10 Unsafe Consumption of APIs: validar respostas da plataforma mãe e não assumir que dados externos são confiáveis.

11. Privacidade e LGPD

- O documento do projeto determina minimização de dados, finalidade explícita, retenção limitada, proteção reforçada de menores e anonimização de demonstrações.
- Para o protótipo, coletar somente o que é necessário para demonstrar o treino. Evitar armazenar vídeo bruto quando métricas/eventos forem suficientes.
- Se imagens de pessoas forem capturadas, definir finalidade, base legal aplicável, controle de acesso, retenção e descarte. Para demonstrações acadêmicas, preferir enquadramento sem pessoas ou anonimização quando possível.

12. Conclusão

- A Sweet Spot possui uma superfície de ataque distribuída: aplicação/API, dados, processamento de câmera, comunicação com IoT e acesso físico ao protótipo.
- As principais prioridades de segurança são controle de acesso, validação de entradas, proteção da API, autenticação de dispositivos, integridade dos comandos, proteção de segredos, disponibilidade do processamento e parada segura.
- A equipe deve integrar os controles do pacote de referência ao código real e repetir os testes com o sistema efetivamente entregue. O relatório não declara vulnerabilidades como corrigidas no projeto real sem evidência de execução.

13. Referências

- Projeto Interdisciplinar - Smart Tennis Arena, FECAP, 6º CCOMP, 2026.
- OWASP Top 10 - documentação oficial; versão atualmente indicada pelo projeto OWASP como 2025, com 2021 como versão anterior: <https://owasp.org/projects/Top-Ten>
- OWASP API Security Top 10 2023: <https://api-security.owasp.org/editions/2023/en/0x11-t10/>
- OWASP Application Security Verification Standard (ASVS), versão estável indicada como 5.0.0: <https://owasp.org/projects/asvs>